



Thought for 1m 49s >

まずひとまとめの結論から：

1. **Decision Console** - マーケ施策意思決定ページは残す前提で、旧 **v1** を完全に置き換える。
2. データはすべてバックエンド集計から取得し、「Invalid Date」「約0.0万円」「ダミーデータ」は一切禁止。
3. Claude には「画面仕様+API仕様+集計SQL/Pythonスケルトン」を一括で渡して、迷いなく実装させる。

以下をそのまま Claude に渡してください（長いですが、このまま 1 メッセージで貼る想定です）。

0. コンテキスト

- 対象プロダクト：CQ0x (Causal Query Optimizer)
- 対象画面：`Decision Console - マーケ施策意思決定`
- ゴール：既存の「今週のベストシナリオ Δ¥ / A/B テスト候補のみ」の簡易画面を ****完全に置き換える正式版コンソール**** を実装すること。
- 前提：
 - ダミーデータ・ランダム生成・フロント側のみの計算は ****禁止****。
 - すべてバックエンド(API)で集計し、フロントは ****表示専用****。
 - 「Invalid Date」「約0.0万円」のような不自然な表示は禁止。
 - 値がない場合は ` ` を表示、極小値なら近似ラベルを出さない。

1. 画面の役割と UX の要点

1.1 役割

「Decision Console - マーケ施策意思決定」は、プロダクト全体の ****マーケティングの司令塔**** として：

- 直近の ****総増分収益 (Total Incremental Profit) ****
- ポリシーあたりの平均 Δ¥
- 因果品質 (CAS) の水準
- 下振れリスク (CVaR)

をひと目で把握しつつ、

- どのポリシー／チャンネル／セグメントが効いているか
- どの施策を「Go / Canary / Hold」とすべきか

を決めるためのダッシュボード。

1.2 画面構成 (上から)

1. ****KPI カード 4 枚****

- Total Incremental Profit (円)
- Average Δ¥ / Policy (円)
- Mean CAS (0~1 or 0~100)
- CVaR (worst 10%) (円, 下振れリスク)

2. ****時系列 & ポートフォリオ****

- 左: Δ¥ Trend (週次 or 日次の折れ線／棒グラフ)
- 中: Segment Portfolio (セグメント別の Δ¥ vs CAS など、散布図 or 棒)
- 右: Channel Performance (チャンネル別 Δ¥ & ROI)

3. ****Decision Cards テーブル****

- 行: 1 施策 (= 1 policy × dataset × run)
- 主な列: Policy, Channel, Segment, Δ¥, ROI, CAS, Verdict, Date
- Verdict フィルタ: All / Go / Canary / Hold
- 行クリックで詳細 (Policy Lab / Causal Diagnostics) へ遷移できるように。

4. ****日付レンジ切替 (右上 or KPI 上部)****

- プリセット: `Last 14 days` / `Last 28 days` / `Last 56 days`
- 選択変更ごとに API を叩き直す。

2. バックエンド API 仕様

2.1 エンドポイント

新規に以下のエンドポイントを追加するか、類似の既存エンドポイントを差し替える：

- `GET /api/v1/decision-console/summary`

クエリパラメータ

- `from` (必須, ISO8601 日付 `YYYY-MM-DD`)
- `to` (必須, ISO8601 日付 `YYYY-MM-DD`)

例:

`/api/v1/decision-console/summary?from=2025-11-01&to=2025-11-28`

レスポンス型

````jsonc

```
{
 "total_delta_yen": 5200000, // 指定期間の増分収益合計
 "avg_delta_yen_per_policy": 2450000, // 施策ごとの平均 Δ¥
 "mean_cas": 0.87, // 施策ごとの CAS 平均 (なければ null)
 "cvar_yen_p10": -300000, // 下振れリスク (最悪10%の期待値) なければ null

 "trend": [
 { "bucket": "2025-W45", "delta_yen": 2200000 },
 { "bucket": "2025-W46", "delta_yen": 3000000 }
],

 "segment_portfolio": [
 {
 "segment_label": "High-LTV Email",
 "total_delta_yen": 3200000,
 "mean_cas": 0.9,
 "mean_risk_score": 0.12 // 必要なら。なければ省略/ null
 }
],

 "channel_performance": [
 {
 "channel": "email",
 "total_delta_yen": 4000000,
 "roi": 5.2,
 "mean_cas": 0.88
 },
 {
 "channel": "app_push",
 "total_delta_yen": 1200000,
 "roi": 3.1,
 "mean_cas": 0.82
 }
],

 "decisions": [
 {
 "id": "analysis-001", // 一意 ID
 "dataset_label": "Web Traffic Oct 2024",
 "policy_name": "Email Campaign Optimization",
 "channel": "email",
 "segment_label": "High-LTV users",
 "delta_yen": 2450000,
 "roi": 2.8,
 "cas_score": 0.87,
 "verdict": "go", // "go" | "canary" | "hold"
 "decided_at": "2025-11-15T00:00:00Z" // ISO8601
 }
]
}
```

```
]
}
```

- `trend.bucket` は週単位 (YYYY-ww) で OK。日次 (YYYY-MM-DD) にしたければ、そのように統一する。
- 値が計算不可能な場合は `null` を返す (フロントで - 表示)。

### 型定義 (Python Pydantic 例)

```
from datetime import datetime, date
from typing import List, Optional
from pydantic import BaseModel

class TrendPoint(BaseModel):
 bucket: str
 delta_yen: float

class SegmentPoint(BaseModel):
 segment_label: str
 total_delta_yen: float
 mean_cas: Optional[float] = None
 mean_risk_score: Optional[float] = None

class ChannelPoint(BaseModel):
 channel: str
 total_delta_yen: float
 roi: Optional[float] = None
 mean_cas: Optional[float] = None

class DecisionRow(BaseModel):
 id: str
 dataset_label: str
 policy_name: str
 channel: Optional[str] = None
 segment_label: Optional[str] = None
 delta_yen: float
 roi: Optional[float] = None
 cas_score: Optional[float] = None
 verdict: str
 decided_at: datetime

class DecisionConsoleSummary(BaseModel):
 total_delta_yen: float
 avg_delta_yen_per_policy: float
 mean_cas: Optional[float] = None
 cvar_yen_p10: Optional[float] = None
 trend: List[TrendPoint]
 segment_portfolio: List[SegmentPoint]
 channel_performance: List[ChannelPoint]
 decisions: List[DecisionRow]
```

### FastAPI ハンドラ例 (スケルトン)

```
from fastapi import APIRouter, Depends, Query
from sqlalchemy.orm import Session
from datetime import date

router = APIRouter()

@router.get("/api/v1/decision-console/summary", response_model=DecisionConsoleSummary)
def get_decision_console_summary(
 from_: date = Query(..., alias="from"),
 to_: date = Query(..., alias="to"),
 db: Session = Depends(get_db_session),
):
 """
 - from_~to_ の期間の completed な因果分析結果のみを集計する。
 - ここではロジックの骨組みだけを書くので、実際のテーブル名/カラム名に合わせて修正すること。
 """
```

```

1) Decision rows を取得
decisions = fetch_decisions(db, from_, to_) # 下に擬似コードを書く

if not decisions:
 return DecisionConsoleSummary(
 total_delta_yen=0.0,
 avg_delta_yen_per_policy=0.0,
 mean_cas=None,
 cvar_yen_p10=None,
 trend=[],
 segment_portfolio=[],
 channel_performance=[],
 decisions=[],
)

2) KPI 集計
total_delta = sum(d.delta_yen for d in decisions)
avg_delta = total_delta / len(decisions)

cas_values = [d.cas_score for d in decisions if d.cas_score is not None]
mean_cas = sum(cas_values) / len(cas_values) if cas_values else None

CVaR: 下位10%の平均
ここでは delta_yen の分布から擬似的に計算 (実際にはシミュレーション結果があればそれを使う)
deltas_sorted = sorted(d.delta_yen for d in decisions)
if len(deltas_sorted) >= 10:
 k = max(1, int(len(deltas_sorted) * 0.1))
 tail = deltas_sorted[:k]
 cvar = sum(tail) / len(tail)
else:
 cvar = None

3) トレンド
trend = aggregate_trend(decisions)

4) セグメント / チャネルの集計
segment_portfolio = aggregate_by_segment(decisions)
channel_performance = aggregate_by_channel(decisions)

return DecisionConsoleSummary(
 total_delta_yen=total_delta,
 avg_delta_yen_per_policy=avg_delta,
 mean_cas=mean_cas,
 cvar_yen_p10=cvar,
 trend=trend,
 segment_portfolio=segment_portfolio,
 channel_performance=channel_performance,
 decisions=decisions,
)

```

## Decision 取得用 SQL (例)

実際のスキーマに合わせて修正する前提のテンプレート:

```

-- decision_console_decisions.sql

SELECT
 a.id AS analysis_id,
 d.name AS dataset_label,
 p.name AS policy_name,
 p.channel AS channel,
 p.segment_label AS segment_label,
 a.delta_yen AS delta_yen,
 a.roi AS roi,
 a.cas_score AS cas_score,
 a.verdict AS verdict, -- 'go' | 'canary' | 'hold'
 a.completed_at AS decided_at
FROM analysis_runs a
JOIN datasets d ON a.dataset_id = d.id

```

```
JOIN policies p ON a.policy_id = p.id
WHERE a.status = 'completed'
AND a.completed_at >= :from
AND a.completed_at < :to + INTERVAL '1 day';
```

トレンド集計例 (PostgreSQL の週単位) :

```
SELECT
 to_char(date_trunc('week', a.completed_at), 'IYYY-W"IW') AS bucket,
 SUM(a.delta_yen) AS delta_yen
FROM analysis_runs a
WHERE a.status = 'completed'
AND a.completed_at >= :from
AND a.completed_at < :to + INTERVAL '1 day'
GROUP BY 1
ORDER BY 1;
```

### 3. フロントエンド実装仕様 (React / TypeScript 前提)

#### 3.1 データ取得

- ページマウント時に、デフォルトで `Last 28 days` を選択し、上記 API を叩く。
- 日付レンジボタンを切り替えるたびに再取得。
- ローディング中はシマー表示、エラー時はページ上部に明示的なエラーバナー。

#### 3.2 通貨・日付フォーマット

##### 通貨

```
function formatYen(value: number): string {
 return `¥${value.toLocaleString("ja-JP", { maximumFractionDigits: 0 })}`;
}

// 補助ラベル (約〇〇万円) は値が十分大きいときのみ表示
function formatYenApprox(value: number): string | null {
 const abs = Math.abs(value);
 if (abs < 10000) return null; // 1万円未満ならラベルなし
 const man = abs / 10000;
 return `約${man.toFixed(1)}万円`;
}
```

- 小さい値で「約0.0万円」とは絶対に表示しない。

##### 日付

```
import { format, parseISO } from "date-fns";
import { ja } from "date-fns/locale";

function formatDecisionDate(iso: string): string {
 try {
 const d = parseISO(iso);
 if (Number.isNaN(d.getTime())) return "-";
 return format(d, "yyyy-MM-dd", { locale: ja });
 } catch {
 return "-";
 }
}
```

- `Invalid Date` は絶対に画面に出さないこと。

#### 3.3 コンポーネント構成案

```
// src/pages/DecisionConsole.tsx

export default function DecisionConsolePage() {
 // 1) 日付レンジ state
 // 2) API 呼び出し (TanStack Query など)
 // 3) ローディング / エラー処理
```

```

return (
 <PageLayout title="Decision Console - マーケ施策意思決定">
 <HeaderFilters ... /> // 日付レンジ切替
 <KpiRow ... /> // 4枚のKPIカード
 <ChartsRow ... /> // Δ¥トレンド / セグメント / チャネル
 <DecisionTable ... /> // 下部テーブル（フィルタ付き）
 </PageLayout>
);
}

```

#### KPI カード

- props: value: number, subtitle?: string, label: string, riskLevel?: "low" | "medium" | "high" など。
- CAS が低い場合は色を変えるなどの視覚的ヒントを入れてもよい。

#### Δ¥ Trend

- X 軸: bucket ( 2025-W45 など を W45 表示 )
- Y 軸: Δ¥
- 棒 or 折れ線グラフ ( 既存チャートライブラリに合わせる )。

#### Segment Portfolio / Channel Performance

- Segment: 縦軸 Δ¥, 横軸 CAS or Risk, バブルサイズ = 件数、など簡易な散布図。
- Channel: 棒グラフでチャネルごとの total\_delta\_yen を表示。色は既存テーマに合わせる。

#### Decision Cards テーブル

- 列:
  - Policy
  - Channel
  - Segment
  - Δ¥ (formatYen)
  - ROI (小数第二位程度)
  - CAS (0~1 or 0~100、プロダクトで決まっている形式を踏襲)
  - Verdict (色付きバッジ: Go=緑, Canary=黄, Hold=赤)
  - Date (formatDecisionDate)
- 上部フィルタ:
  - Verdict: All / Go / Canary / Hold トグルボタン
  - Search: treatment/outcome 名での部分一致フィルタ (必要なら)
- 行クリック時:
  - policy\_name + dataset\_label + analysis\_id を元に、Policy Lab や Diagnostics へ navigation。
  - ルーティング仕様は既存実装に合わせる。

## 4. エッジケースとバリデーション

### 1. データがない期間を選択した場合

- KPI はすべて 0 or -。
- グラフは空の状態 ( 「データがありません」 を表示 )。
- テーブルは 「No decisions in selected period」。

### 2. CAS や CVaR が計算できない場合

- バックエンドで null を返し、フロントは - を表示。
- 「0」ではなく null を使うことで、計算不能と真の 0 を区別する。

### 3. API エラー

- ページ上部にエラーバナー: 「Decision Console データの取得に失敗しました」。
- 再試行ボタンを配置してもよい。

### 4. セキュリティ

- 外部 API やトラッキングへの送信は行わない。
- 既存の認証／認可ミドルウェアをそのまま利用。

## 5. 実装方針の確認事項

- この仕様は、既存の簡易 Decision Console を **置き換える**。
  - 追加の新ページは作らない。
  - ルートパス `/console`（または現状のパス）をこの新 UI にマッピング。
- 既存 API を破壊的に変更する必要がある場合は、**新しい v1 エンドポイント** を追加し、フロント側をそちらに切り替えること。
- ダミーデータ・ランダム生成・「とりあえず見た目だけ」の実装は **禁止**。  
すべて上記 API レスポンスからレンダリングすること。

```

What this does (この回答がやること)

- Decision Console を「今週のベストシナリオ」レベルから、**世界標準のマーケ投資ダッシュボード**に引き上げるための、Claude 向け完全仕様書です。
- バックエンド API のパラメータ・レスポンス構造・集計ロジックの骨格と、フロント側のコンポーネント構成・フォーマット規則を、**ダミーデータ禁止・Invalid Date 禁止**まで含めて明示しています。
- これをそのまま Claude に渡せば、「どの値をどこからどう計算するのか」「どんな UI になるべきか」をブレなく実装できます。

Expert insight (Google/Meta/NASA レベル)

トップ企業の計測系ダッシュボードは、**「1 画面 = 1 つの明確な意思決定」**を徹底しています。
今回の Decision Console も「直近 X 日のマーケ投資の収益性とリスクを見て、どの施策を打つか決める」という 1 本の問いに絞り込んでいるので、ここに余計な指標や
別目的のグラフを足し過ぎない方が、結果的に「月額 1,000 万払う価値がある」画面になります。

::contentReference[oaicite:0]{index=0}
```